pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

The Framework

WEEK 2 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: the framework

Days 2 to 8: your own code

`tool_schema`	the docstring, read
`agent_loop`	request, tools, repeat
`run_tools`	the message bookkeeping
`check_figures`	every figure, checked

Today

1. What a framework is for
2. Tracing off, and why
3. function_tool beside tool_schema
4. sdk_agent on the course's client
5. Day 2's question both ways: body_diff
6. await in a notebook
7. sdk_trace_table beside trace_table
8. What it does not do
9. LangGraph in one paragraph

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

What an agent produces: text, every number from a tool, whoever builds the request

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

On the synthetic book and the curve: numbers from your tools, never a view

Allowed names

DV01 per 100 of face

position DV01 in dollars

the largest DTS contributions in the file

Never called

cheap, rich, or attractive

a trade idea or a recommendation

overweight or underweight

How a framework works

The problem	Every agent needs schemas, a loop, a stop rule, and the message list; a framework supplies them once
The objects	Agent: a name, instructions, tools, a model, settings. Runner.run: an agent and an input
The result	final_output and every item of the run; no check on the answer
You control	The instructions, the tools, the settings, and the step limit
What comes out	A result object. What does not come out is any check, which stays yours

What a framework is for

	Days 2 to 8	The framework
Tool schemas	tool_schema	function_tool
The loop	agent_loop	Runner.run
The stop rule	max_steps	max_turns
Bookkeeping	run_tools, tool_message	its own items
Checks on the answer	check_figures, yours	none

Story: the package's own record

openai-agents on PyPI: 0.0.1 uploaded 2025-03-04, yanked ("Updating to real release"); 0.0.2 on 2025-03-11.

The course pins 0.23.1, uploaded 2026-10-02; PyPI lists 123 versions with files.

Sources (read 2026-10-11): pypi.org/pypi/openai-agents/json; api.github.com/repos/openai/openai-agents-python

A framework writes four of the five parts. The fifth, the checks, stays yours.

Tracing off, and why

```
os.environ["OPENAI_AGENTS_DISABLE_TRACING"] = "1"   # before import agents: read once, at start
import agents
agents.set_tracing_disabled(True)                     # after: a manual setting wins from then on
```

NoOpTrace

what a new trace is now

two

switches, so import order cannot undo it

The default exporter, read in agents/tracing/processors.py:

`https://api.openai.com/v1/traces/ingest`

Nothing about a run leaves the machine unasked, and the saved runs hold only the agent's requests.

function_tool beside tool_schema

function_tool(dv01): the framework's schema


```
{"properties": {"settlement": {"description": "the settlement date as YYYY-MM-DD, for example 2026-10-02
.", "title": "Settlement", "type": "string"}, "maturity": {"description": "the maturity date as YYYY-MM-
DD.", "title": "Maturity", "type": "string"}, "coupon_pct": {"description": "the annual coupon in percen
t, 5.0 for a 5% coupon.", "title": "Coupon Pct", "type": "number"}, "yield_pct": {"description": "the yi
eld to maturity in percent, 5.06 for 5.06%.", "title": "Yield Pct", "type": "number"}}, "required": ["se
ttlement", "maturity", "coupon_pct", "yield_pct"], "title": "dv01_args", "type": "object", "additionalPr
```


tool_schema(dv01): day 1's


```
{"type": "object", "properties": {"settlement": {"type": "string", "description": "the settlement date a
s YYYY-MM-DD, for example 2026-10-02."}, "maturity": {"type": "string", "description": "the maturity dat
e as YYYY-MM-DD."}, "coupon_pct": {"type": "number", "description": "the annual coupon in percent, 5.0 f
or a 5% coupon."}, "yield_pct": {"type": "number", "description": "the yield to maturity in percent, 5.0
6 for 5.06%."}}, "required": ["settlement", "maturity", "coupon_pct", "yield_pct"], "additionalPropertie
```


The framework adds titles and nothing else. Equal once the titles are taken out: True.

How sdk_agent works

The problem	The framework runs one object, an Agent, and the course needs it built one way
The assembly	Course instructions, tools as function tools, a model on the course's client, the course's settings
The client	Every request leaves through the recording transport, so a saved run replays without a key
You control	The name, the agent's own instructions, the tools, the output type, and the day
What comes out	An agents.Agent that Runner.run can run

sdk_agent on the course's client

```
desk_agent = agentkit.sdk_agent("desk", "Answer with the tool.", [dv01], aclient, agentkit.MODELS["desk"], 9)
```

```
instructions start with DESK_PROMPT: True; its own sentence: 'Answer with the tool.'
temperature 0.0, parallel_tool_calls False, store False, extra_args {'seed': 42, 'prompt_cache_key': 'cours...
tools ['dv01'], parameters sent ['settlement', 'maturity', 'coupon_pct', 'yield_pct']
```

How the framework chooses store, in its own source:

```
default_store = True if cls.is_openai(client) else None
return model_settings.store if model_settings.store is not None else default_store
```

On OpenAI's own address, store is true unless the settings say otherwise; elsewhere it is not sent.

Day 2's question, both ways: body_diff

body_diff(loop, framework)

field	in_a	in_b
messages	True	True
prompt_cache_key	False	True
store	False	True
tools	True	True

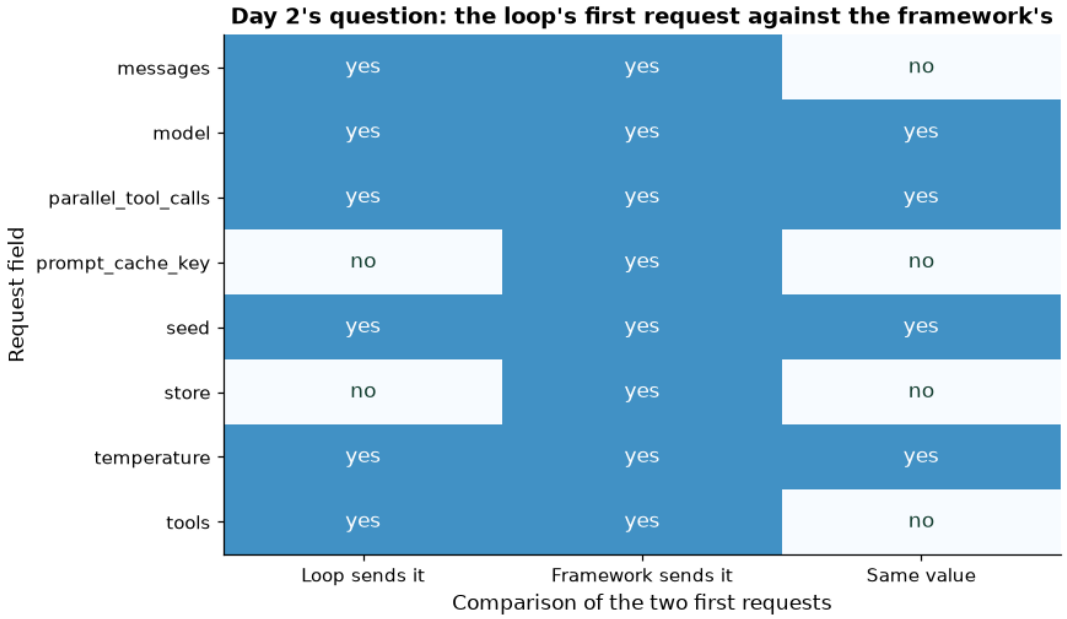
```
store: False; prompt_cache_key: 'course8-day09'
system message: DESK_PROMPT + '\nAnswer with the tool.'
the user message is the same: True
the tools are the same once the titles are taken out: True
temperature as sent: the loop 0, the framework 0.0
the two first requests have the same exchange key: no
```

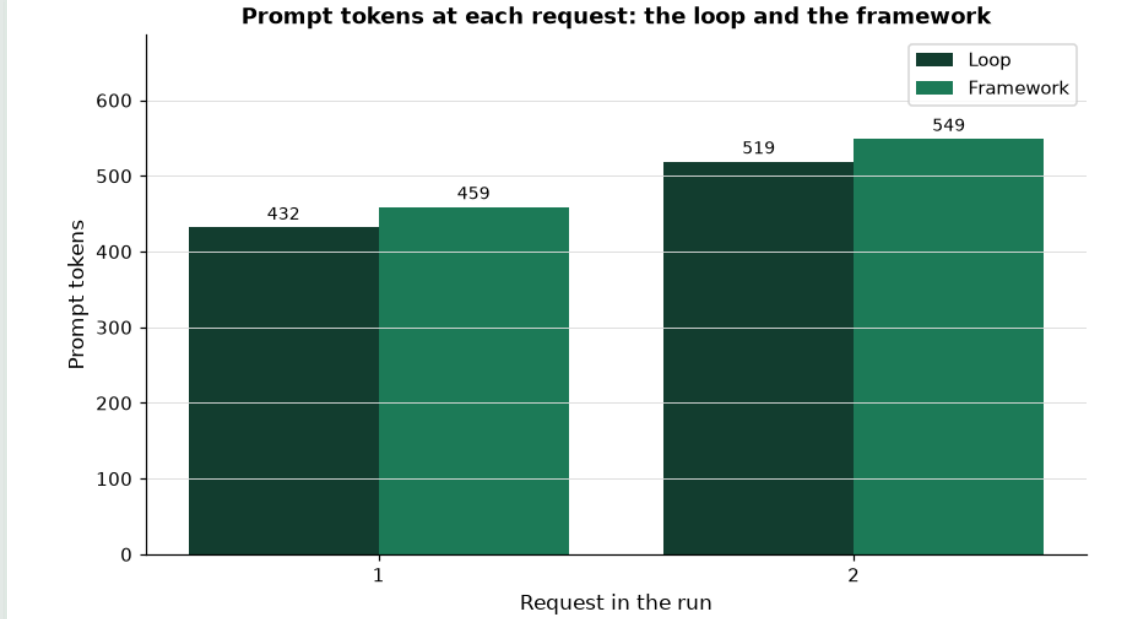
Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 951 prompt and 104 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 1,008 prompt and 126 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

Added: prompt_cache_key and store. Temperature 0 against 0.0: equal as numbers, other bytes, another key.

The two first requests, drawn





Left: the light cells are what the framework adds or changes. Right: its extra sentence.

Same question, same tool, same answer: the requests differ in the framework's own fields.

Why sdk_settings sets two fields

```
default_agent = agents.Agent(..., model=agents.OpenAIChatCompletionsModel(model=..., openai_client=fake_defaults))
await agents.Runner.run(default_agent, QUESTION, max_turns=8)        # twice, each in its own cell
```

```
run 1: store True; prompt_cache_key begins agents-sdk:run: True
run 2: store True; prompt_cache_key begins agents-sdk:run: True
the two runs sent the same prompt_cache_key: no
the same question, the same exchange key: no
requests to a traces path: 0
```

store=False

the response is not kept

course8-day09

the same key on every run

A new random key per run changes every body, so every exchange key: a saved run is never found again.

How Runner.run works

The loop	Send a request, run the tools the reply asks for, add their results, and send again
The end	A final answer, or max_turns reached and MaxTurnsExceeded raised
Asynchronous	Calling it returns a coroutine that does nothing until it is awaited
You control	The input and the step limit; later a session, hooks, and a context
What comes out	A result with final_output and the items of the run

await Runner.run in a notebook

```
outcome = await agents.Runner.run(desk_agent, QUESTION, max_turns=8)   # a notebook: one await per cell
asyncio.run(main())                                                    # a script of your own
Runner.run_sync(...)        # the scan refuses it in a notebook; its source says it cannot run here
```

```
asyncio.run: asyncio.run() cannot be called from a running event loop
requests sent by this cell: 0
```

coroutine

a run described, not started

one await

per cell: one failed request, one cell

A notebook already runs an event loop: await hands the run to it.

sdk_trace_table beside trace_table

```
the loop, trace_table:
 step role tool arguments finish_reason prompt_tokens completion_tokens
 0 user <NA> <NA>
 1 assistant dv01 {...} tool_calls 432 42
 1 tool dv01 <NA> <NA>
 2 assistant stop 519 62
the framework, sdk_trace_table:
 step role tool arguments finish_reason prompt_tokens completion_tokens
 0 user <NA> <NA>
 1 assistant dv01 {...} tool_calls 459 42
 1 tool dv01 <NA> <NA>
 2 assistant stop 549 84
the same columns: True
```

The same columns, read from the bodies the client sent: every check of the course works on both.

What the framework does not do: check a number

```
market_agent = agentkit.sdk_agent("market desk", ..., output_type=agentkit.DeskAnswer)
agentkit.check_figures(outcome_market.final_output, run_market)
```

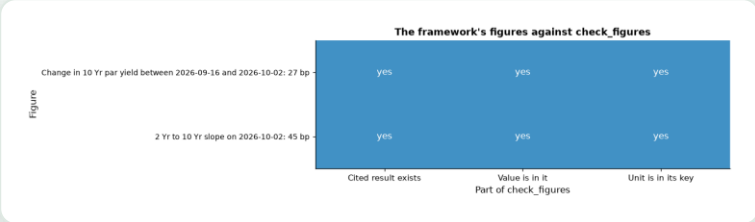
```
Change in 10 Yr par yield between 2026-09-16 and 2026-10-02 27.0 bp 27.0 True True True True
2 Yr to 10 Yr slope on 2026-10-02 45.0 bp 45.0 True True True True
```

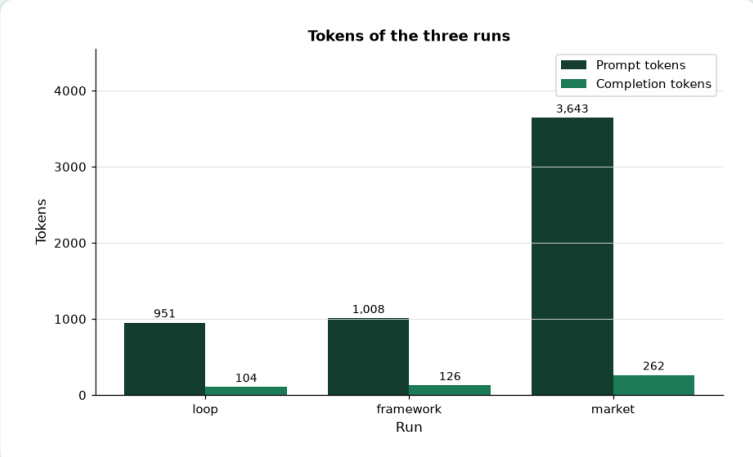
Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 3 requests, 3,643 prompt and 262 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

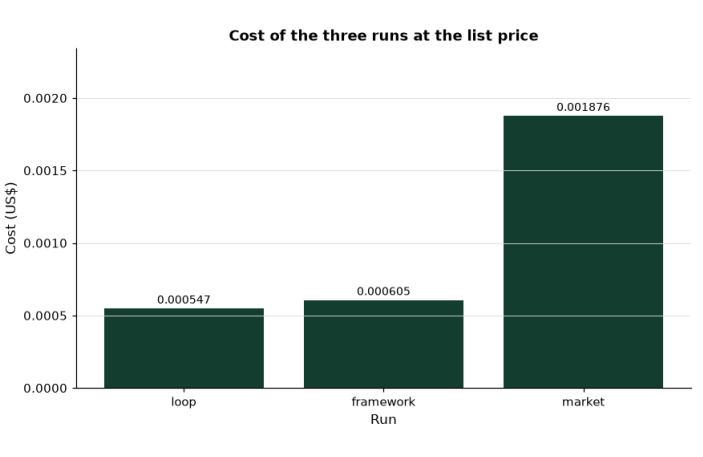
```
loop       numbers no tool returned: []
framework  numbers no tool returned: []
```

The framework checks the JSON's shape. Whether a figure is its tool's value: your code, after the run.

What the three runs used and cost







Run	Requests	Tool requests	Prompt	Completion	Cost (US$)
loop	2	1	951	104	0.000547
framework	2	1	1,008	126	0.000605
market	3	2	3,643	262	0.001876

Every field of an Agent

Field	Default	Course sets	Tune first	What it controls
`name`	required	yes	-	the agent's name, shown in a trace and to o...
`tools`	list()	yes	yes	the function tools the model may request
`instructions`	None	yes	yes	the system message
`model`	None	yes	-	the model object that sends the requests
`model_settings`	get_default_m...	yes	yes	temperature, tool choice, and the other req...
`output_type`	None	yes	yes	the shape of a structured final answer; Non...

16 fields, the course sets 6. Handoffs come on day 11, guardrails on day 14.

Every field of ModelSettings

Field	Default	The course sets	Tune first	What it controls
`temperature`	None	0.0	yes	how random the choice of each word is...
`tool_choice`	None	-	yes	whether the model must, may, or may n...
`parallel_tool_calls`	None	False	-	whether the model may request several...
`max_tokens`	None	-	yes	the most tokens in one reply
`store`	None	False	yes	whether the provider keeps the response
`extra_args`	None	{'seed': 42, 'promp...	yes	extra request fields passed as they a...

25 fields, the course sets 4: temperature, parallel_tool_calls, store, and the seed and cache key in extra_args.

Every setting of Runner.run

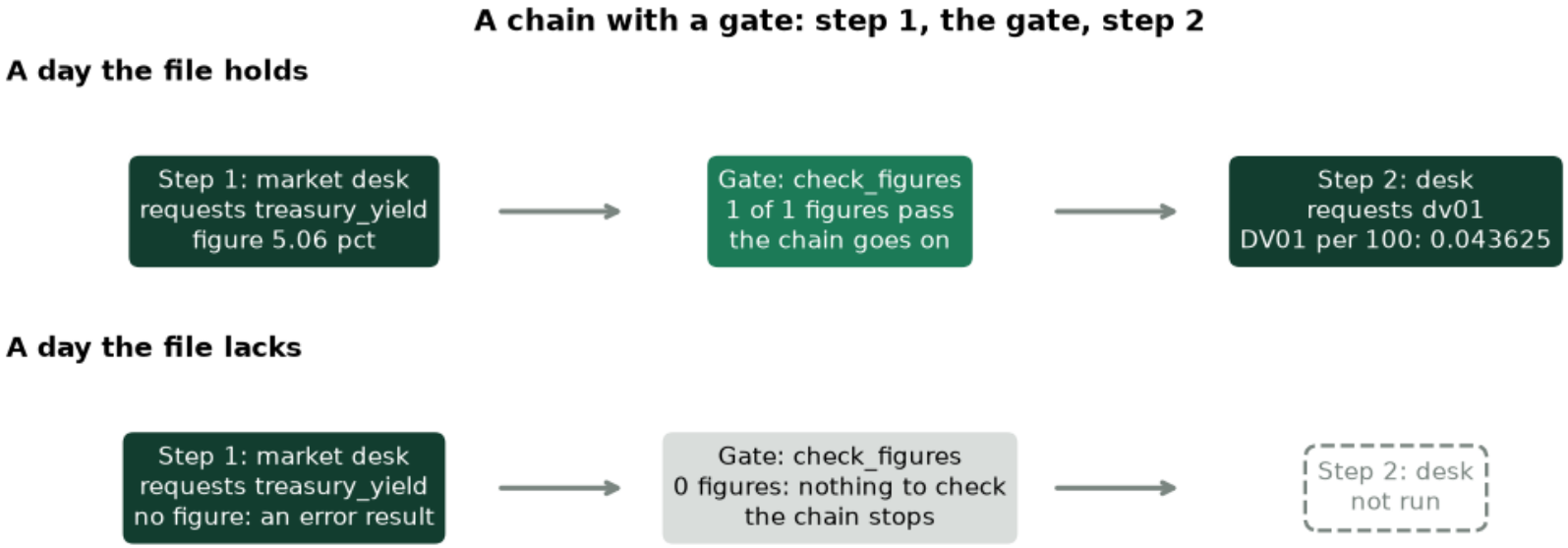
Setting	Default	The course passes	Tune first	What it controls
`starting_agent`	required	the agent	-	the agent that takes the first turn
`input`	required	the question	-	the question, or a list of messages
`max_turns`	10	8	yes	the most requests before MaxTurnsExceeded...
`hooks`	None	day 14	-	callbacks on the run's events (day 14)
`session`	None	day 12	-	a stored conversation the run reads and e...

11 settings. Every cell of the course passes max_turns=8: the cap on cost and on a run that never answers.

How prompt chaining works

The problem	Some tasks are a fixed series of steps, each needing the one before
The chain	Separate agent runs with ordinary Python between them; step 1 returns a structured answer
The gate	A function that checks step 1's answer: a figure stated, every figure the value of its cited result
You control	The steps, the gate, and what passes from one step to the next
What comes out	The last step's run, or step 1's run and a stopped chain: no later request is sent

A chain with a gate

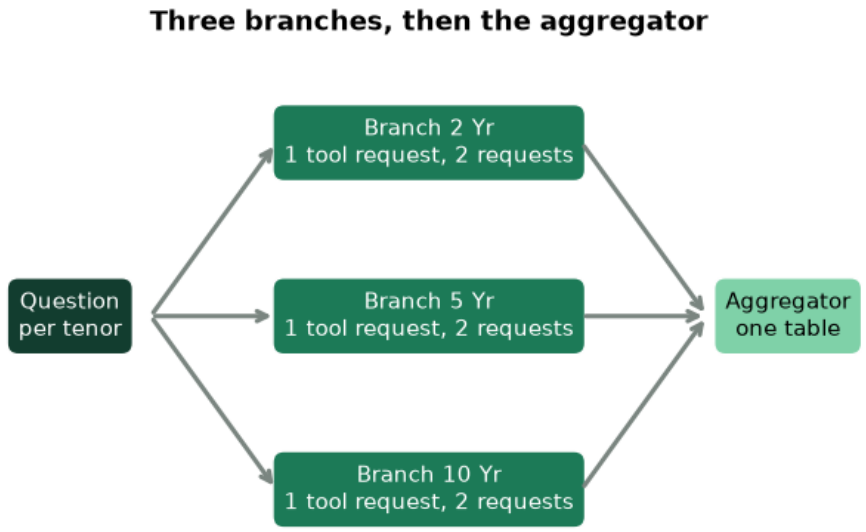


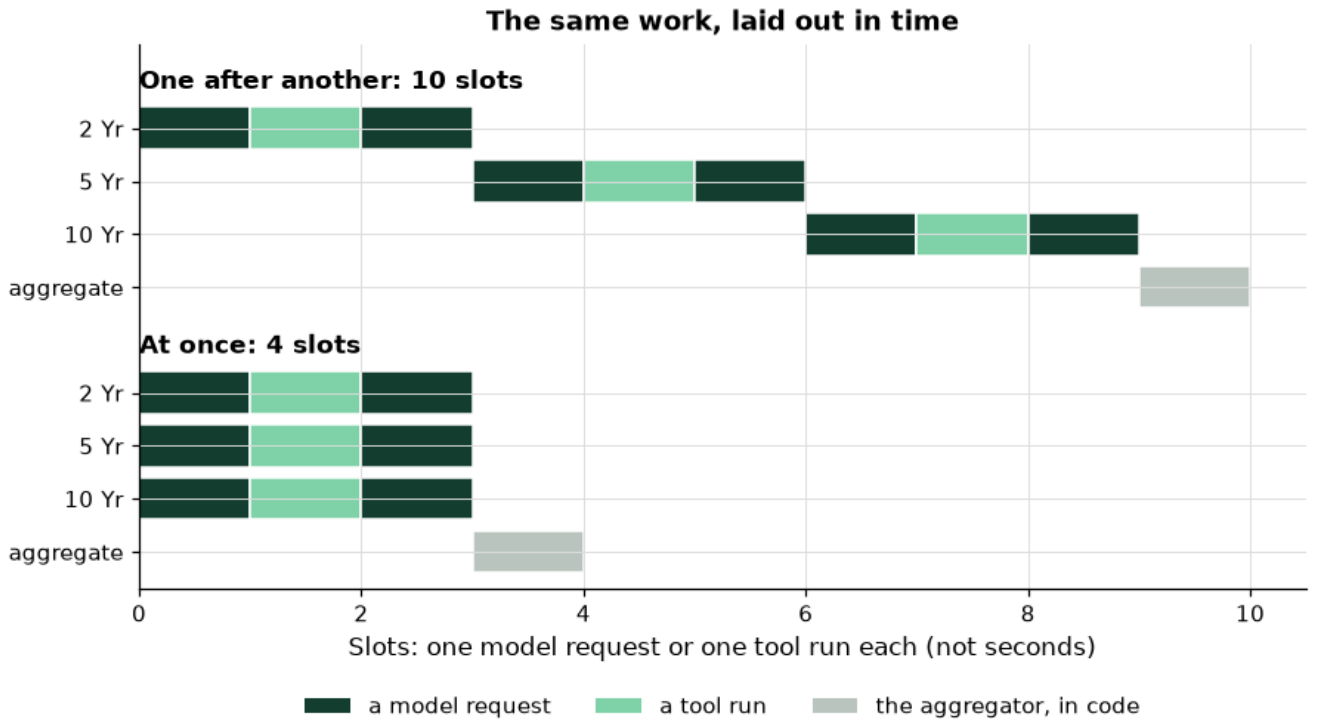
a day in the file: step 1 requested ['treasury_yield']; gate passed: yes; step 2 ran: yes

How parallelization works

The problem	The parts of a task do not depend on each other, so they need not wait for each other
The fork	One run for each part, awaited together with asyncio.gather; each branch has its own client
The aggregator	Code that joins the results: here one table, every figure checked against its own run
You control	How many branches, the question each gets, and how they are joined
What comes out	One result in the time of the slowest branch, for the same requests as one after another

Three branches, then the aggregator



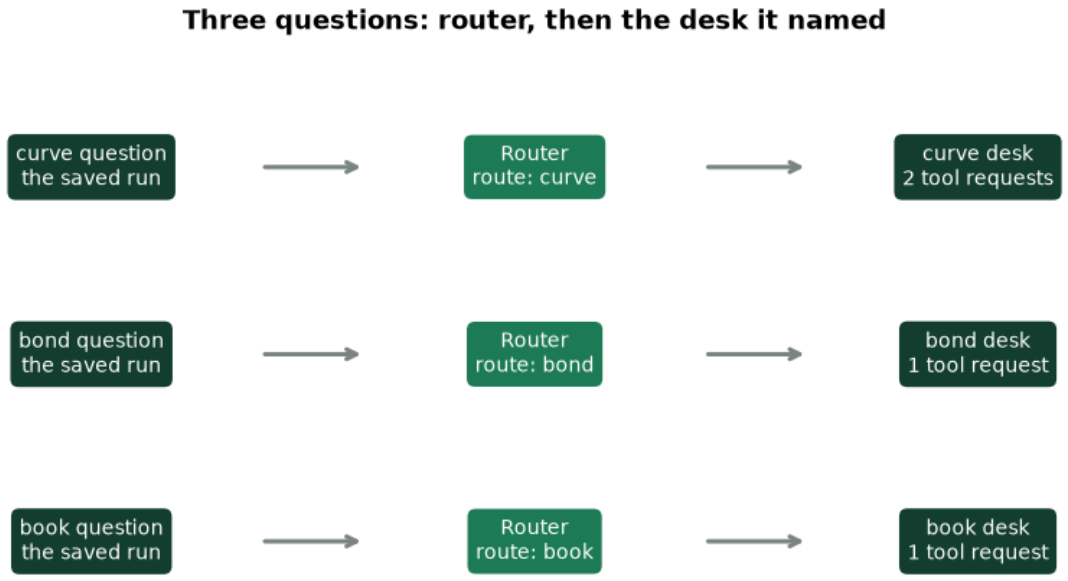


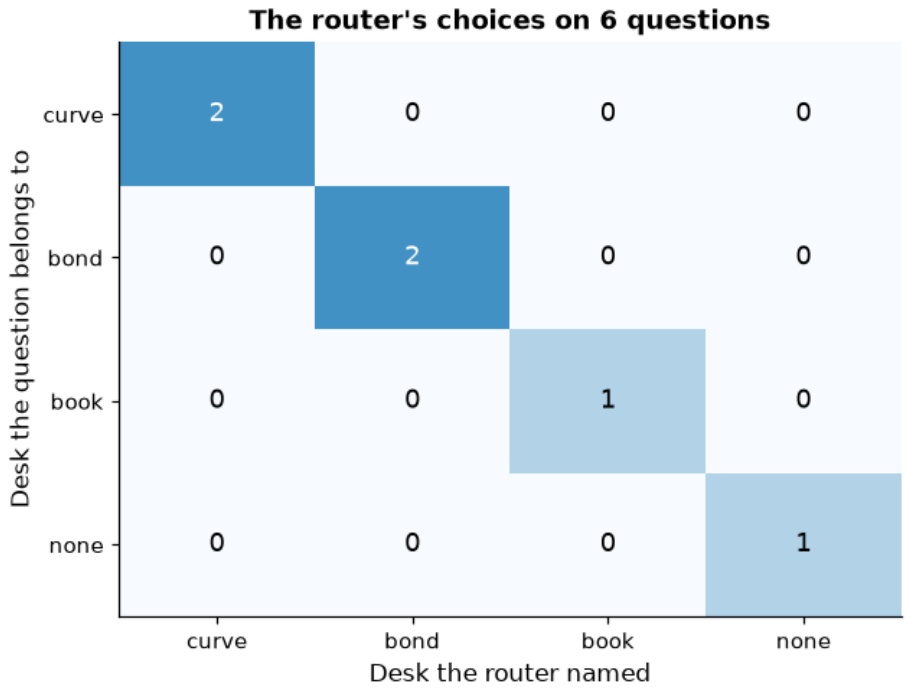
10 slots one after another, 4 at once: steps counted, not seconds measured.

How routing works

The problem	Requests of different kinds are best answered by agents with their own tools
The router	An agent with no tools; its output is one of four names, in a model that can hold only those
The dispatch	Your code reads the name and sends the question to the desk that owns it; none sends it nowhere
You control	The names, the router's instructions, and which agent each name leads to
What comes out	One small request for the route, then the chosen desk's own run

Routes and the router's choices



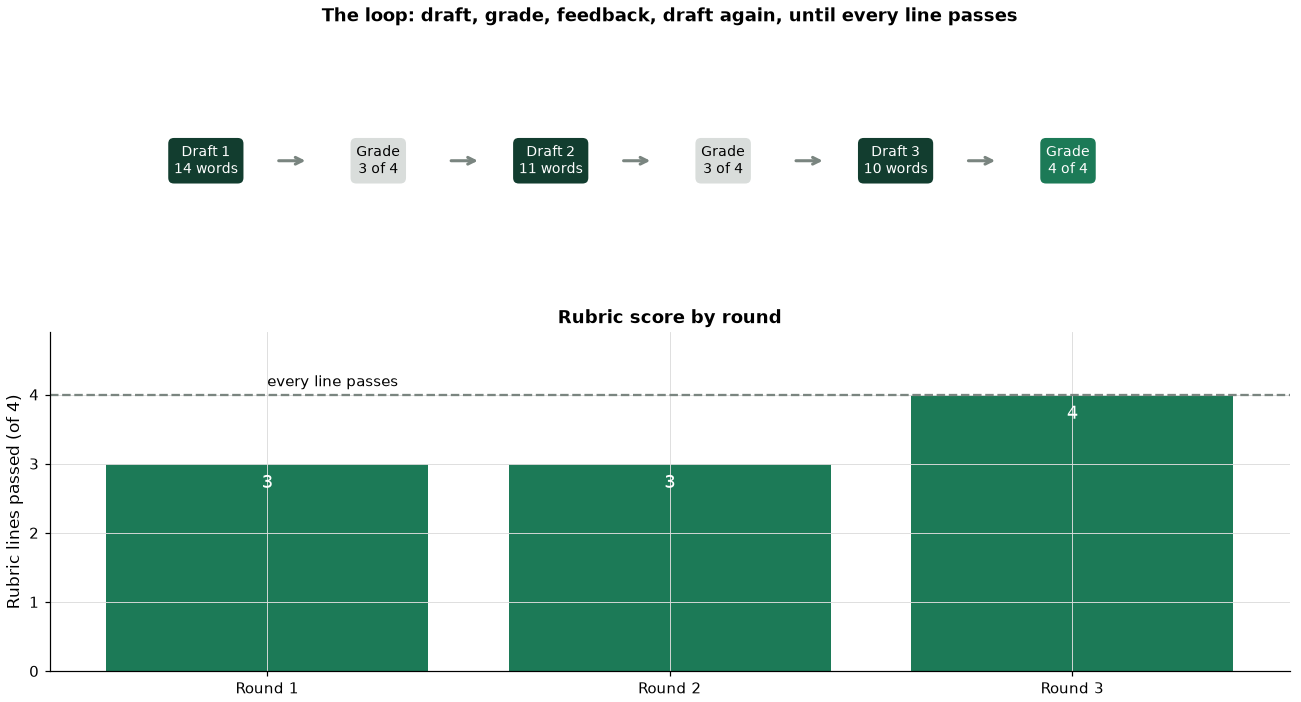


6 of 6 questions named the desk they belong to; the off-scope question got none and no desk ran.

How the evaluator loop works

The problem	A first draft is rarely the one you want, and you can often say what is wrong with it
The loop	The drafter writes; the evaluator grades against a rubric of yes-or-no lines; failed lines go back as feedback
The cap	A limit on rounds ends a loop that never passes, so the worst case is known
You control	The rubric, the feedback, and the cap
What comes out	The last draft, each round's grade, and a request count with a known worst case

The loop and the rubric score by round



3 rounds, rubric lines passed [3, 3, 4] of 4: the word limit was the line that failed.

Workflow or agent

Pattern	Requests	The code checks	Use it when
Chain with a gate	4	the gate: check_figures on step 1	each step needs the one before, and a bad step 1 must s...
Parallelization	6	the aggregator: every branch's fi...	the parts do not depend on each other, and time matters
Routing	13	the route is one of four names	requests fall into kinds, and each kind has its own tools
Evaluator and optimizer	4	the rubric, line by line	you can write down what a good answer is, and a first d...
An agent (days 2 to 8)	2	check_figures after the run	the path cannot be written down ahead of time

Who chooses the next step: your code in a workflow, the model's replies in an agent.

LangGraph, Excel, and Git

LangGraph, as the course probed it on 2026-10-08

langgraph 1.2.14, langchain-core 1.6.7: MIT, released 2026-10-06, about 19.7 MB

Its nodes are your code: request bodies byte-identical to the raw loop's; replayed as shipped

LangSmith tracing off unless its variables are set. Not installed in the course.

Excel: no counterpart today. No agent framework, no tool requests; today's numbers were tested on days 1, 5, 6, 8.

```
agentkit.py | 144 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
test_agentkit.py | 87 ++++++++++++++++++++++++++++++++
2 files changed, 231 insertions(+)
```

git diff --stat HEAD~1 HEAD in the notebook; git diff --stat week1 HEAD in my_bondmath. 50 passed.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

Tracing off, store false, the fixed key course8-day09.

→

2

Code back

Written for the course in the style of an AI answer. It answers.

→

3

Test first

No trace processor, a fake server, one run. It fails on store.

→

4

Fix

Tracing off; sdk_agent with sdk_settings inside. It passes.

Today: make_agent switches tracing on and leaves the defaults: store true, a random cache key every run.

Code that runs is not the same as code that is right.

What you can do now

- A framework writes schemas, the loop, the stop rule; not the checks
- Tracing off, twice, before and after import agents
- body_diff: the framework adds store and prompt_cache_key
- sdk_settings: store false, a fixed key, so runs replay
- One await per cell; check_figures on the framework's run

Week 2, Day 5 Case study: a macro agent on two statements